

CockroachDB

Day 2: Distributed Operations & Resilience

 Transactions · Query Tuning · Multi-Region ·
Throughput

From "It Works" to "It Works Under Load"



Day 2 Schedule

Module 2: Distributed Operations & Resilience

5. Transactions & Consistency  Lab 5 · 65 min

6. Query Optimization  Lab 6 · 60 min

7. Multi-Region Topologies & Resilience  Lab 7 · 65 min

8. High-Volume Schema Patterns

9. Throughput Engineering  Lab 8 · 60 min

Today's through-line: every performance problem you meet — contention, slow plans, cross-region latency — traces back to a schema decision. Lab 8 puts numbers on all of them.

5. Transactions & Consistency

SERIALIZABLE — Always, By Default

CockroachDB's default isolation level is SERIALizable. There is no READ COMMITTED default and no setting to silently weaken it.

What this means in practice:

- Concurrent transactions behave *as if* they ran one at a time
- No dirty reads, no non-repeatable reads, no phantom reads, no write skew
- You pay for it with transaction retries under contention
- Application code must handle retries gracefully

As of v23.2, READ COMMITTED is available opt-in — but the course defaults assume SERIALizable.

MVCC — Many Versions, No Locks for Readers

Every key in CockroachDB is stored with a timestamp. A read at time T returns the most recent version at or before T .

MVCC key layout (simplified)

```
/users/'abc'/email @ T=1000 → "old@example.com" (committed)
/users/'abc'/email @ T=2000 → "new@example.com" (committed)
/users/'abc'/email @ T=2500 → "newer@example.com" (write intent - uncommitted)
```

- Readers don't block writers — a read at $T=1500$ sees the $T=1000$ version
- Writers don't block readers — same reason
- Write-write conflicts resolve via Raft and transaction ordering

Write Intents & Conflict Detection

When a transaction writes a key, it leaves a write intent — a provisional value tagged with the transaction's ID and timestamp.

Two transactions touching the same row

```
Txn A: BEGIN; UPDATE balance SET amt = amt - 100 WHERE id = 'x';  -- intent written
Txn B: BEGIN; UPDATE balance SET amt = amt + 50 WHERE id = 'x';

→ Txn B sees A's intent. Txn B WAITS (or pushes A's timestamp).
→ When A commits, its intent becomes a committed MVCC version.
→ Txn B then re-reads and proceeds — possibly at a later timestamp.
```

If serializability would be violated, the conflict is resolved by retrying one of the transactions with a fresh read timestamp. The client sees a retryable error.

Retryable Errors — Your App Must Handle Them

SQLSTATE 40001 — serialization_failure

```
ERROR: restart transaction: TransactionRetryWithProtoRefreshError:  
WriteTooOldError: write at timestamp ... too old; wrote at ...  
SQLSTATE: 40001
```

The retry rule: the client must run the entire transaction again, from `BEGIN` to `COMMIT`. Not just the last statement.

- Most drivers ship a retry helper (e.g. `crdb.run_transaction()` in `psycopg`)
- Most ORMs need a wrapper — see the integrations section
- The cluster will also do internal retries for many short, single-statement transactions before surfacing an error

Follower Reads & AS OF SYSTEM TIME

For read-only queries that can tolerate slightly stale data, you can read from any replica, not just the leaseholder:

Two flavors

```
-- Exact-time historical read (default ~4.8s in the past)
SELECT * FROM orders AS OF SYSTEM TIME follower_read_timestamp();

-- Read at a specific HLC timestamp
SELECT * FROM orders AS OF SYSTEM TIME '2026-05-22 10:00:00';

-- Whole transaction at a single past timestamp (useful for reporting)
BEGIN;
SET TRANSACTION AS OF SYSTEM TIME follower_read_timestamp();
SELECT ... FROM big_table JOIN ...;
COMMIT;
```

- Cuts cross-region latency dramatically — read from the local replica
- Skips contention with concurrent writes — historical reads can't conflict
- Tradeoff: data is stale (typically by 4-5 seconds)

SELECT FOR UPDATE — Pessimistic Locking

SERIALIZABLE is optimistic: conflicts are detected at commit and the loser retries. When contention is high, that means work is done and thrown away repeatedly.

```
BEGIN;  
SELECT balance FROM accounts WHERE id = $1 FOR UPDATE; -- lock now  
UPDATE accounts SET balance = $2 WHERE id = $1;  
COMMIT;
```

Optimistic (default)

- No locks taken
- Wins at low contention
- Conflicts cost a full retry

FOR UPDATE

- Writers queue instead of racing
- Wins at high contention
- Locks add latency and deadlock risk

Before either: can the read-modify-write collapse into *one* statement (UPDATE ... SET n = n + 1)? Then it's atomic server-side and needs neither locks nor retries. Lab 14 benchmarks all three.

Contention Anti-Patterns

Anti-Pattern	Symptom	Fix
Single-row counter (<code>UPDATE counters SET n = n + 1</code>)	All writes contend	Sharded counter or message queue
Long-running read tx blocking writes	Write intents pile up	Use <code>AS OF SYSTEM TIME</code> for reports
<code>SELECT * + UPDATE</code> on same connection (no <code>BEGIN</code>)	No atomicity, repeated retries	Wrap in explicit <code>BEGIN ... COMMIT</code>
Bulk insert through one connection	Sequential commits, slow	Parallelize across connections, use multi-row <code>INSERT</code>



Lab 5: Transactions, Contention & Retry Loops



65 min

Objectives

- Trigger a 40001 retry from two concurrent sessions
- Implement a working retry loop in `scripts/crdb sql` or `psql`
- Use `AS OF SYSTEM TIME` to bypass contention for a report
- Spot contention in the DB Console's Transactions dashboard



Setup

```
scripts/crdb up # 3-node cluster in Docker
```

Lab walks you through opening *two* connected sessions side by side.

Lab Activities

- Part A: Two-session contention demo — produce a 40001
- Part B: Wrap the transaction in a retry loop
- Part C: Switch the read to `AS OF SYSTEM TIME` and observe the contention disappear

6. Query Optimization

Three Levels of EXPLAIN

Command	What it shows	Runs the query?
EXPLAIN	Plan tree + estimated row counts	No
EXPLAIN ANALYZE	Plan + actual times, rows, network I/O	Yes
EXPLAIN ANALYZE (DEBUG)	Produces a downloadable bundle for support	Yes

Always start with `EXPLAIN ANALYZE` — estimated rows are useful, but actual rows tell you where the optimizer was wrong.

Reading an EXPLAIN ANALYZE

Annotated output

```
EXPLAIN ANALYZE SELECT name, total FROM orders
WHERE customer_id = '550e8400-...' AND status = 'open';

planning time: 0.4ms
execution time: 2.1ms           ← total wall-clock
distribution: local
vectorized: true

• filter
  estimated row count: 3
  actual row count: 2           ← matches estimate (good!)
  filter: status = 'open'
  |
  └─ . scan
      nodes: n2                 ← where it ran
      regions: us-east1
      actual row count: 12
      KV time: 1.4ms            ← bulk of the cost
      table: orders@orders_customer_id_idx
      spans: ['/550e8400-...' - '/550e8400-...'] ← single-key span
```

Statistics & the Cost Model

- CockroachDB collects table statistics automatically (sampled, periodic)
- Stats include row count, distinct values per column, null %, histograms
- The optimizer uses stats to estimate selectivity and pick the cheapest plan
- After bulk loads or large schema changes, refresh stats manually:

Refreshing & inspecting stats

```
-- Force a refresh on one table
CREATE STATISTICS my_stats FROM orders;

-- See what's there
SHOW STATISTICS FOR TABLE orders;

-- Per-column histogram
SHOW HISTOGRAM <histogram_id>;
```



Lab 6: EXPLAIN ANALYZE & Query Tuning



60 min

Objectives

- Read an EXPLAIN ANALYZE plan and identify the dominant cost
- Use the Insights page to find a missing-index recommendation
- Apply STORING, partial, and composite indexes to fix specific plans
- Watch the optimizer change its mind after `CREATE STATISTICS`



Setup

```
docker-compose up # 3-node cluster in Docker  
# Lab loads a ~100k-row orders dataset before tuning.
```

Lab Activities

- Part A: Profile three queries with EXPLAIN ANALYZE; predict the bottleneck before reading the plan
- Part B: Add indexes one by one; measure the change in actual time and KV time
- Part C: Use the SQL Activity → Insights tab to find a recommendation

7. Multi-Region Topologies & Resilience

Multi-Region — Three Building Blocks

1. Localities — every node is tagged with `region`, `zone`, etc.
2. Survival goal — what level of failure should the database survive without going read/write unavailable?
3. Table locality — where do the rows of a table live, and where are their leaseholders?

Get these three right and CockroachDB does the heavy lifting (replica placement, leaseholder placement, follower reads) automatically.

Localities — Tagging Nodes

Set at node startup — a flag, not SQL

```
cockroach start \  
  --locality=region=us-east1,zone=us-east1-b,rack=r4 \  
  --join=...
```

Lab 7's demo `--global` assigns them for you — read them back with `SELECT node_id, locality FROM crdb_internal.gossip_nodes;`

- Localities are hierarchical — left-to-right, broadest to narrowest
- The cluster uses them to spread replicas across failure domains
- Most multi-region clusters use at least `region` and `zone`
- Misconfigured localities = your "multi-region" cluster may have every replica in one region

Defining the Cluster's Regions

SQL setup

```
-- Tell the database which regions it's deployed in
ALTER DATABASE bank SET PRIMARY REGION "us-east1";
ALTER DATABASE bank ADD REGION "us-west1";
ALTER DATABASE bank ADD REGION "europe-west1";

-- Inspect
SHOW REGIONS FROM DATABASE bank;
```

The primary region is the default home for new tables. `ADD REGION` adds more — each region must already have nodes running with the matching locality tag.

How a Region Becomes Placement

Three layers, one join key: the region *name*. Nothing places data directly — each layer writes a request the next one satisfies.

```
node      --locality=region=europe-west1,zone=b      what the machine advertises
database  ALTER DATABASE shop ADD REGION 'europe-west1'  which advertised regions it uses
table     LOCALITY REGIONAL BY ROW                    how rows are spread among them
|
▼ generates, per partition
zone cfg  constraints = {+region=europe-west1: 1}      a request in locality terms
          lease_preferences = [[+region=europe-west1]]
|
▼ matched continuously by
allocator moves replicas + leases to nodes whose --locality satisfies it
```

```
SELECT node_id, locality FROM crdb_internal.gossip_nodes;  -- what nodes offer
SHOW ZONE CONFIGURATION FROM TABLE customers;             -- what the table asks for
SHOW RANGES FROM TABLE customers WITH DETAILS;           -- where the allocator put it
```

The failure is asymmetric. `ADD REGION` for a name no node advertises is refused on the spot. But a node started with `region=us-east-1` when the database expects `us-east1` is *not* refused — it joins, gets no constrained replicas, and nothing says why. Compare the two queries above when placement looks wrong.

Survival Goals — What Should the DB Survive?

Goal	Tolerates	Replicas needed per range
ZONE FAILURE (default)	Loss of any 1 zone in the primary region	3 (across 3 zones in 1 region)
REGION FAILURE	Loss of any 1 entire region	5 (across at least 3 regions)

Setting the goal

```
ALTER DATABASE bank SURVIVE REGION FAILURE;
```

Tradeoff: REGION FAILURE needs cross-region quorum on every write → write latency includes cross-region RTT. Always-on, but slower writes.

Table Locality — Where Do Rows Live?

Locality	Best for	Read latency	Write latency
REGIONAL BY TABLE	Single-region apps, reference data per region	Local in primary region	Local in primary region
REGIONAL BY ROW	User data with home regions (multi-tenant SaaS)	Local for that row's region	Local for that row's region
GLOBAL	Rarely-changing, read-anywhere reference data	Local everywhere	Cross-region (slow)
Follower reads (any locality)	Reports, dashboards — a few seconds stale is fine	Local everywhere	n/a

The question per table: is this row's audience one region, or all of them? One region → BY ROW. All, rarely written → GLOBAL. All, stale OK → follower reads. All, written everywhere, must be current → *no local option exists*; pin BY TABLE to the busier region and let the other pay.

REGIONAL BY ROW in Practice

Per-row geo-partitioning

```
ALTER DATABASE bank PRIMARY REGION "us-east1";
ALTER DATABASE bank ADD REGION "us-west1";      -- one region per statement
ALTER DATABASE bank ADD REGION "europe-west1";

CREATE TABLE customers (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email   STRING UNIQUE,
  region  crdb_internal_region NOT VISIBLE AS (
    CASE
      WHEN email LIKE '%%.eu' THEN 'europe-west1'
      ELSE 'us-east1'
    END
  ) STORED
) LOCALITY REGIONAL BY ROW AS region;
```

```
CREATE TABLE orders (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id  UUID NOT NULL REFERENCES customers(id),
  region        crdb_internal_region NOT NULL      -- copied from the customer at insert
) LOCALITY REGIONAL BY ROW AS region;
```

Give related tables the same region column. A European customer's rows in `customers`, `orders` and `order_items` are then all led from Europe, and a join between them stays in-region. Each table is still on its own ranges — this is region-level locality, not co-location.

GLOBAL Tables — Read-Anywhere

For reference data (currency tables, product catalogs, country lists) that is read in every region but rarely written:

Global locality

```
CREATE TABLE country_codes (  
  code  STRING(2) PRIMARY KEY,  
  name  STRING NOT NULL  
) LOCALITY GLOBAL;
```

CockroachDB uses non-blocking transactions to make these tables look like they have a leaseholder in *every* region. Reads are fast everywhere; writes are slow (they wait long enough that all regions can safely serve the new value).

Rule of thumb: if you write to it more than ~once an hour per row, GLOBAL is the wrong choice.

The Allocator — Who Decides Where Replicas Live

Every node runs an allocator that continuously answers: *is this range where it should be?*

Input	Effect on placement
Zone configuration	Hard constraint — replica count, region/zone requirements
Node liveness	Dead node's replicas are re-created elsewhere after <code>server.time_until_store_dead</code>
Store capacity	Prefers less-full stores; rebalances away from imbalance
QPS / load	Load-based rebalancing moves leases off hot stores
Locality diversity	Spreads replicas across the failure domains you declared

```
-- Is anything at risk right now?  
SELECT count(*) FROM crdb_internal.ranges_no_leases WHERE array_length(replicas,1) < 3;  
  
-- The queue doing the work (a STORE metric: lives in kv_store_status)  
SELECT node_id, store_id, metrics->>'queue.replicate.pending' AS pending  
FROM crdb_internal.kv_store_status;
```

Under-replicated > 0 for more than a few minutes is a real alert — those ranges are one failure from unavailability. A replicate queue that stays non-zero means the allocator cannot drain its work: usually disk or network, occasionally a zone config that cannot be satisfied.



Lab 7: Multi-Region Topologies & Survival Goals

65 min

Objectives

- Spin up a simulated 3-region, 9-node cluster with a containerised `demo --global` cluster
- Toggle survival between `ZONE FAILURE` and `REGION FAILURE` and measure write latency
- Convert a table to `REGIONAL BY ROW` and verify local reads
- Take a region offline and confirm the cluster keeps serving



Setup

```
docker run --rm -it -p 8090:8090 cockroachdb/cockroach:v23.2.5 \
  demo --global --nodes 9 --no-example-database --empty --http-port=8080
```

Lab Activities

- Part A: Set primary region and survival goal; measure cross-region write latency
- Part B: Apply `REGIONAL BY ROW` to a customers table; query from each region
- Part F: Take Europe offline on a second (non-global) demo cluster; verify availability



8. High-Volume Schema Patterns

The Playbook, applied

Day 1 Claimed It. Today You Measure It.

In a distributed SQL database, the primary key is a distribution decision.
Uniqueness is the easy half.

Everything you've seen today traces back to it:

- Contention (Lab 5) — two transactions fighting over one range
- Slow plans (Lab 6) — a scan that fans out because keys aren't co-located
- Cross-region latency (Lab 7) — rows in the wrong place

Lab 8 puts a rows/sec number on each design, so the argument stops being a matter of taste.

Pattern 1 — Hash-Sharded Time-Series PK

For: write-heavy, timestamp-ordered data — events, metrics, audit logs

```
CREATE TABLE events (  
  account_id UUID,  
  created    TIMESTAMPTZ DEFAULT now(),  
  id         UUID DEFAULT gen_random_uuid(), -- now() is the TXN timestamp:  
  payload    STRING,                        -- without this, one statement's  
  PRIMARY KEY (account_id, created, id)      -- rows collide on the PK  
  USING HASH WITH (bucket_count = 16)  
);
```

Avoids: the rightmost-range write hotspot on monotonic keys.

Costs: ordered scans now fan out across every bucket.

Watch: `bucket_count` $\approx$ your write concurrency. 16 is usually right; over 64 rarely helps and makes range scans worse.

Pattern 2/3 — Append-Only Log & Per-Tenant Co-located PK

```
-- Append-only event log
CREATE TABLE event_log (
  id      UUID PRIMARY KEY
          DEFAULT gen_random_uuid(),
  ts      TIMESTAMPTZ DEFAULT now(),
  payload JSONB
) WITH (ttl_expire_after = '30 days');
```

UUID scatters writes; TTL bounds growth.

```
-- Multi-tenant co-location
CREATE TABLE orders (
  tenant_id UUID,
  order_id  UUID DEFAULT gen_random_uuid(),
  ...
  PRIMARY KEY (tenant_id, order_id)
);
```

One tenant's rows sit together — one range per query, not N.

The catch with co-location: one hot tenant becomes one hot range. Combine with hash sharding for that tenant, or split them out.

Pattern 5 — Sharded Counter

```
-- The anti-pattern: every writer serializes on one row  
UPDATE counters SET n = n + 1 WHERE name = 'page_views';
```

```
CREATE TABLE counter_shards (  
  name STRING, shard INT2, n INT DEFAULT 0,  
  PRIMARY KEY (name, shard)  
);  
  
-- The APPLICATION picks the shard and binds it as a parameter  
UPDATE counter_shards SET n = n + 1  
WHERE name = 'page_views' AND shard = $1; -- $1 = randint(0, 15)  
  
SELECT sum(n) FROM counter_shards WHERE name = 'page_views';
```

The trap: `WHERE shard = (random() * 16) :: INT` looks equivalent and is not. `random()` is volatile and evaluated per row scanned, so the statement matches a random *number* of rows — sometimes zero (the increment vanishes), sometimes several (it double-counts). Measured: 193 and 211 out of 200 for two server-side forms; exactly 200 for the client-chosen shard.

The trade: reads become a 16-row scan instead of a point lookup; writes stop serializing. Under concurrency that's a trade you take every time — and Lab 8 measures exactly how much it buys.



9. Throughput Engineering

Numbers you can quote in a design review

Four Ways to Load Rows — and When to Use Each

Method	Relative speed	Table online?	Use for
Single-row <code>INSERT</code>	1× (latency-bound)	✓	Only when the row arrives alone
Multi-row <code>INSERT</code>	10–100×	✓	Application-driven ingest
<code>COPY</code>	Fast	✓	Large loads that must stay online
<code>IMPORT INTO</code>	Fastest	✗ offline	One-time loads and migrations

Why single-row is so slow: each statement is its own implicit transaction — a full Raft round trip *and* a network round trip per row. You're measuring latency, not throughput.

"Batch Your Inserts" Is Useless Without a Number

Batch size	Typical behaviour
1	Latency-bound, worst case
10 → 100	Steep gains
500 – 2,000	Usually the knee
5,000+	Flat or worse — more intents, more contention, bigger blast radius

Large batches also hit `kv.transaction.max_intents_bytes`, and when a batch fails you retry *the whole batch*. The knee is about failure blast radius as much as it is about speed. Measure yours — Lab 8 Part B.

Pre-Split Before a Bulk Load

```
-- A new table is ONE range. 100M rows go to one leaseholder
-- until the splitter catches up.
ALTER TABLE big_import SPLIT AT
  SELECT g * 100000 FROM generate_series(1, 19) g;

IMPORT INTO big_import (id, payload) CSV DATA ('s3://...');

-- Release the manual splits, or ranges can never merge back
ALTER TABLE big_import UNSPLIT ALL;
```

Sizing the splits: one per ~512 MiB of expected data, or one per node $\times$ 3–10, whichever is larger. You're seeding the allocator, not micro-managing it.

Online Schema Changes Under Load

✓ Background job

- CREATE INDEX
- ADD COLUMN with a default
- DROP COLUMN

Non-blocking, resumable, cancellable. Traffic continues.

⚠ Validating backfill

- ALTER COLUMN TYPE
- Adding NOT NULL to existing data
- Adding a foreign key

Full scan of the table. Schedule it.

```
SELECT job_id, description, status, fraction_completed
FROM [SHOW JOBS]
WHERE job_type IN ('SCHEMA CHANGE', 'NEW SCHEMA CHANGE');
-- 'NEW SCHEMA CHANGE' = declarative changer (most DDL)
-- 'SCHEMA CHANGE'     = legacy changer (e.g. TRUNCATE)
```

Remember the price: every secondary index means another KV write per row, and each one is a Raft proposal.

The Concurrency Knee

```
for C in 1 4 16 64 256; do
  scripts/crdb run workload run kv --duration=30s --concurrency=$C \
    --read-percent=50 'postgresql://root@crdb1:26257?sslmode=disable' | tail -4
done
```

It runs *inside* the cluster network — hence `crdb1`, not `localhost`.

Throughput plateaus long before latency does. The knee is the last concurrency level where throughput still climbs meaningfully. Past it, you're buying latency with no throughput.

Little's Law: $\text{concurrency} = \text{throughput} \times \text{latency}$. 2,000 QPS at 8 ms mean latency needs $2000 \times 0.008 \approx 16$ connections — not 200. An oversized pool moves the queue from your app into the database, where it's harder to see and more expensive to drain.



Lab 8: Throughput Engineering & the Pattern Playbook



60 min

Objectives

- Rank four ingest methods by measured rows/sec
- Find *your* batch-size knee instead of guessing
- Prove the pre-split speedup on a sequential-key table
- Run four PK designs under concurrent load and produce a rows/sec table
- Replace a single-row counter and measure the retry collapse
- Find the concurrency knee and size a pool from it



Setup

```
scripts/orchestrator # 3-node cluster in Docker
# Keep a results table open - the deliverable is the filled-in numbers,
# not the individual commands.
```

Deliverable

A completed results table: ingest rankings, batch knee, pre-split speedup, best/worst PK design, sharded-counter speedup, and a pool size you derived rather than inherited.

Day 2 Wrap-Up

- Transactions — SERIALIZABLE by default; 40001 is the protocol, not a bug
- Query tuning — read the plan, collect statistics, index for the plan you want
- Multi-region — survival goals and table localities are latency and compliance decisions
- Schema patterns — ten named designs, each with the trap it avoids
- Throughput — batching, pre-splits, PK design, and the concurrency knee, all measured

Tomorrow (Day 3): those same numbers become dashboards, alerts, node counts, and a hardware bill — plus backup/DR drills and security hardening.

Appendix — If Time Allows

Deep-dive slides, in teaching order. Not needed to follow the labs.

Retry Loop — Pseudocode

Application-side retry pattern

```
import psycopg2
from psycopg2 import errors

MAX_RETRIES = 5

def transfer(conn, src, dst, amount):
    for attempt in range(MAX_RETRIES):
        try:
            with conn:
                with conn.cursor() as cur:
                    cur.execute(
                        "UPDATE accounts SET balance = balance - %s WHERE id = %s",
                        (amount, src))
                    cur.execute(
                        "UPDATE accounts SET balance = balance + %s WHERE id = %s",
                        (amount, dst))

            return # COMMIT successful
        except errors.SerializationFailure:
            # 40001 - retry the entire transaction
            continue
    raise RuntimeError("transfer failed after retries")
```

Common mistake: catching and ignoring the error. The transaction is already aborted — the next statement on that connection will fail until you start fresh.

Index Hints & Index Recommendations

Usually the optimizer picks well. When it doesn't, you have two levers:

Hint a specific index

```
SELECT * FROM orders@orders_customer_id_idx
WHERE customer_id = 'abc';

-- Force a full table scan (rarely useful, but available)
SELECT * FROM orders@{NO_INDEX_JOIN}
WHERE customer_id = 'abc';
```

CockroachDB will tell you when you're missing an index. Check the SQL Activity → Insights tab in the DB Console — it surfaces queries that would benefit from a new index, and even gives you the `CREATE INDEX` statement.

DistSQL & Vectorized Execution

The plan tells you *where* the work runs and *how* it is executed.

```
EXPLAIN (DISTSQl) SELECT category, count(*) FROM products GROUP BY category;
EXPLAIN (VEC)      SELECT category, count(*) FROM products GROUP BY category;
```

Plan line	Means
distribution: full	Work is spread across nodes — good for big aggregations, waste for a point lookup
distribution: local	Runs on the gateway node
vectorized: true	Column-at-a-time execution — much faster for scans and aggregates

The diagnostic that matters: `distribution: full` on a query that should touch one tenant means your keys are not co-located. That is a schema finding, not a query finding.

```
-- A downloadable bundle: plan, statistics, trace, environment
EXPLAIN ANALYZE (DEBUG) SELECT ...;
```

Common Slow-Query Patterns

Symptom in EXPLAIN ANALYZE	Likely cause	Fix
Full table scan, span [-]	Missing index for predicate	Add covering index
Index join shows >> rows than final result	Index is too narrow	Add <code>STORING</code> clause
Estimated rows << actual rows	Stale statistics	<code>CREATE STATISTICS</code> manually
<code>distribution: full</code> for a small query	Cross-range scan	Narrow predicate or co-locate via composite PK
High <code>KV time</code> vs execution time	Reading lots of keys to filter little	Better index, partial index, or <code>STORING</code>

Node Decommissioning & Recovery

Removing a node cleanly

```
# Tell node 4 to drain — it stops being a leaseholder and rebalances replicas off
scripts/crdb run node decommission 4 --insecure

# Watch progress
scripts/crdb run node status --insecure --decommission
```

`scripts/crdb run <args>` is just `cockroach <args>` executed inside node 1 — which is why the production form you will find in the docs works unchanged.

- Decommissioning is online — no downtime, no read/write interruption
- The cluster waits until *every* range has a replacement replica before marking the node decommissioned
- Use this for: scale-down, hardware retirement, planned maintenance, region evacuation

Recovery From an Unplanned Loss

What CockroachDB does automatically

```
T+0    Node 3 disappears (kernel panic, network partition, ...)
T+9s   Lease on Node 3's ranges expires; peers acquire new leases
T+5min Allocator decides: too few replicas, start re-replicating
        Each affected range starts streaming a snapshot to a new node
T+...  When every range is back to 3 replicas, the cluster is healthy again
```

Operator response: usually none, unless the node was permanently lost.
Use the DB Console's Replication dashboard to confirm under-replicated ranges return to zero.

Patterns 6–10 — The Rest of the Playbook

#	Pattern	For	Avoids
6	Hot/Cold split via partial index	99% of queries hit a recent slice	Indexing rows nobody queries
7	GLOBAL reference table	Small read-anywhere lookup data	Cross-region reads on hot paths
8	Outbox + CDC	Publishing events atomically with writes	Dual-write inconsistency
9	TTL table	Sessions, tokens, soft locks	DELETE jobs competing with live traffic
10	Pre-split + bulk import	One-time loads of sequential data	Writing 100M rows into one range

Full catalog with schemas, traps, and where each is taught: [labs/SCHEMA_PATTERNS_PLAYBOOK.md](#). Days 3 and 4 keep applying them — capacity math, migration redesign, outbox implementation.

The Anti-Patterns to Spot in Code Review

Spot this	Because
<code>id SERIAL PRIMARY KEY</code> on a high-volume table	Rightmost-range hotspot
<code>UPDATE counters SET n = n + 1 ...</code>	40001 retry storms
Low-cardinality lead column (<code>region, status</code>)	Huge ranges, poor selectivity
"Index everything just in case"	Each index multiplies every insert
<code>DELETE FROM logs WHERE created < ...</code> on a cron	Competes with live traffic — use TTL
App writes to Kafka <i>and</i> the database	Dual write — use Outbox + CDC
Read-then-write with no transaction	No atomicity guarantee at all